Chapter 8
Monte Carlo Methods for the Propagation of Uncertainties



8.1 Introduction to Monte Carlo Methods

Random sampling to investigate physical and mathematical problems has been in use for several centuries (De Comte Buffon 1774; Kelvin 1901). However, the first practical use, and coining of the term "Monte Carlo" (after the casino in Monaco) for the method, date to Los Alamos National Laboratory during the Manhattan Project. The Monte Carlo method was first proposed for studying diffusion of neutrons into radiation shielding in 1947, and the first computerized Monte Carlo calculations were carried out on the Electronic Numerical Integrator and Computer (ENIAC) in 1948 (Metropolis 1987; Richtmyer and Von Neumann 1947). It was not until computers became more powerful and more available in the 1960s and 1970s that Monte Carlo methods started gaining widespread traction as a way to simulate various engineering problems.

The reliance on computers is due to the very nature of Monte Carlo methods: drawing conclusions from numerous trial calculations using large quantities of random numbers. Applying the method requires each input variable in an equation to be randomly sampled from some probability distribution. The sets of sampled input variables are combined to determine one possible output value. Repeated enough times, statistical conclusions can be drawn from the set of calculated possible outputs. According to The Law of Large Numbers (Graham and Talay 2013), as the number of samples approaches infinity, the expected output value approaches its true value. Because of the large quantity of calculations and random numbers required, early Monte Carlo methods computed by hand were impractical.

Example—Calculating Pi A classic use of Monte Carlo is calculating the value of π. Knowing that the ratio of a square's area to the area of an inscribed quarter-circle is $\pi/4$, points can be sampled at random from within the square to see how many fall inside the circle. By counting the dots where $\sqrt{x^2 + y^2} \leq 1$, the ratio of points inside the circle to the total number of points can be computed. Figure 8.1 shows three cases, with increasing numbers of random sampled points. When multiplied by 4, the

© Springer Nature Switzerland AG 2020

S. Crowder et al., *Introduction to Statistics in Metrology*,
https://doi.org/10.1007/978-3-030-53329-8_8

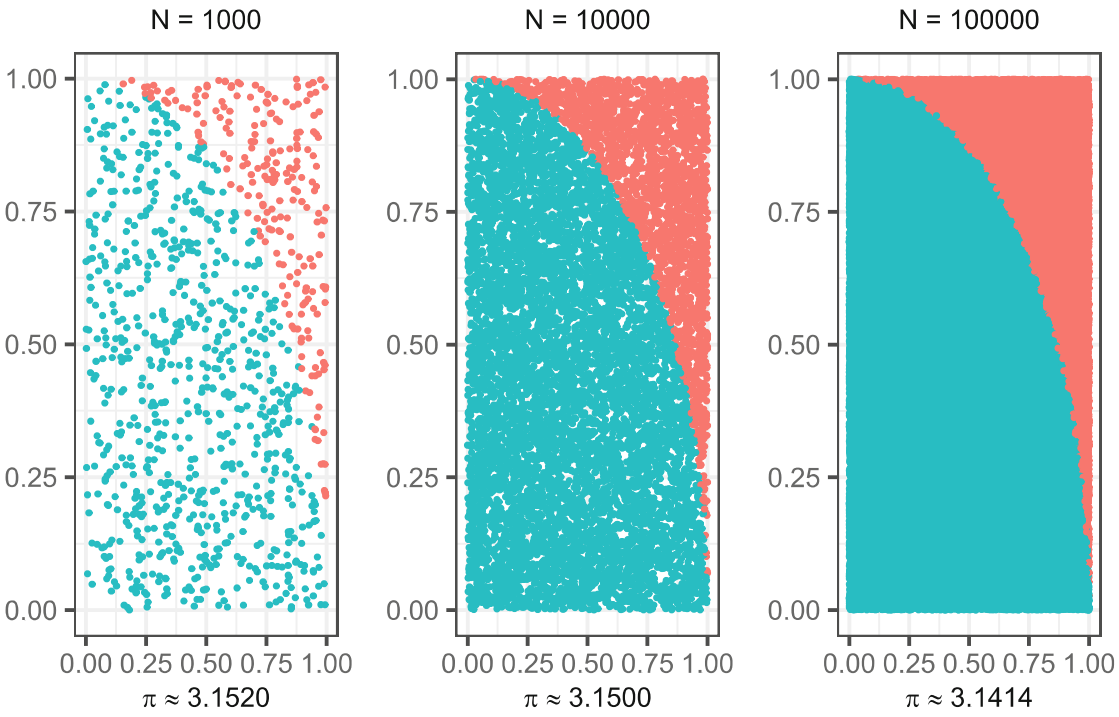


Fig. 8.1 Calculation of π using Monte Carlo method with 1,000 (left), 10,000 (middle), and 100,000 (right) samples

result for this trial with $N = 100{,}000$ was 3.1414, or approximately π. As seen in the figure, the more points sampled, the closer this approximation will come to the true value of π.

8.1.1 Random Sampling Techniques and Random Number Generation

Because the Monte Carlo method relies on random sampling from a distribution, a good random number generator is an essential, but often overlooked, requirement. While precursors to Monte Carlo analysis created randomness by physical means, such as rolling dice, computer algorithms are required to efficiently obtain a sufficient quantity of random values. Most modern computer programming languages or statistical packages already implement algorithms for random number generation and sampling from distributions, but the basic concepts are described below.

Computerized random number generators fall into two categories: true-random number generators and pseudo-random number generators (PRNGs). True random number generators produce completely random, independent, unpredictable samples. These generators are essential in applications such as encryption, where predictability in the random sequence could mean finding a way to break the encryption. True random generators are almost impossible to implement completely within software. PRNGs, on the other hand, appear random, yet follow a prescribed

algorithm. They are predictable if the initial state of the generator and details of the algorithm are known. In engineering applications such as Monte Carlo analysis, the repeatability of a PRNG is desirable for debugging and testing the algorithms. PRNGs are also much simpler and can be implemented completely within software. The remainder of this section discusses PRNGs and their use in Monte Carlo analysis.

Several properties of a PRNG should be evaluated for its suitability to Monte Carlo analysis:

1. *Sequence Length* PRNGs are essentially state machines where the next random number depends on the previous output and the internal state of the generator, meaning a PRNG can only generate a specific number of samples before repeating itself. Obviously, this sequence length must be longer than the number of samples required. Due to possible long-term hidden correlations between the sections of the output, it is recommended that the sequence length must be at least the square of the required number of samples (Hellekalek 1998). For typical uncertainty propagation problems, 1 to 10 million samples for each variable are required to converge on a solution. By this metric, for a simple problem with 3 variables needing 10 million samples each, the PRNG must be able to generate at least ~2^{50} (~10^{15}) samples before repeating (sequence lengths are frequently reported as a power of 2). For problems with many input variables, this number should be even higher.
2. *Uncorrelated Output* The generated values must be independent and uncorrelated. Many algorithms appear independent over short sequences of numbers yet contain some amount of long- or short-term order to the values they produce. Reducing the number of samples used from the sequence as described above can mitigate the longer-term correlation effects.
3. *Uniform Generation* A PRNG must generate samples uniformly over its range to be useful for simulation. In other words, it should be unbiased in producing its output values.
4. *Reproducibility and Portability* For Monte Carlo and other engineering applications, the same sequence of numbers should be generated at different times and on different platforms. This is to aid in debugging and testing the simulation and model.

For some recommended algorithms that meet these requirements, and an assessment of the PRNGs in popular programming languages, see Sect. 8.1.3.

8.1.1.1 Sampling from Normal and Non-Normal Distributions

Most PRNG algorithms generate a uniform distribution of numbers, usually in the range of 0–1. However, in Monte Carlo analysis, normal or other distribution shapes are required. Transformations are necessary to convert the uniform random numbers into the required distribution. The simplest transformation takes the uniform

distribution on the interval [0, 1] and applies a shift and scale to move the uniform distribution to the desired range.

The Box–Muller transformation (Box and Muller 1958), for example, transforms two uniformly distributed independent random samples U_1 and U_2 into two normally distributed independent samples Z_1 and Z_2:

$$\begin{aligned} Z_1 &= \sqrt{-2 \ln U_1} \cos(2\pi U_2) \\ Z_2 &= \sqrt{-2 \ln U_1} \sin(2\pi U_2) \end{aligned} \tag{8.1}$$

Similar transformations have been developed to produce other distribution types; see Robert and Casella (2004) for examples.

8.1.1.2 Generating Correlated Random Samples (Normal Distribution)

Correlated random variables are generated using similar transformations. These transformations take two or more uniform random samples and a covariance matrix and convert them into correlated random variables of another distribution. The multivariate normal distribution is the most common of these transformations. For implementation details, see Gentle (2009) and Cox and Harris (2006).

When generating correlated random samples, it is often useful to generate pairwise scatter plots of the samples. If there is slope or other order to the scatter points, the two variables are correlated (Fig. 8.2).

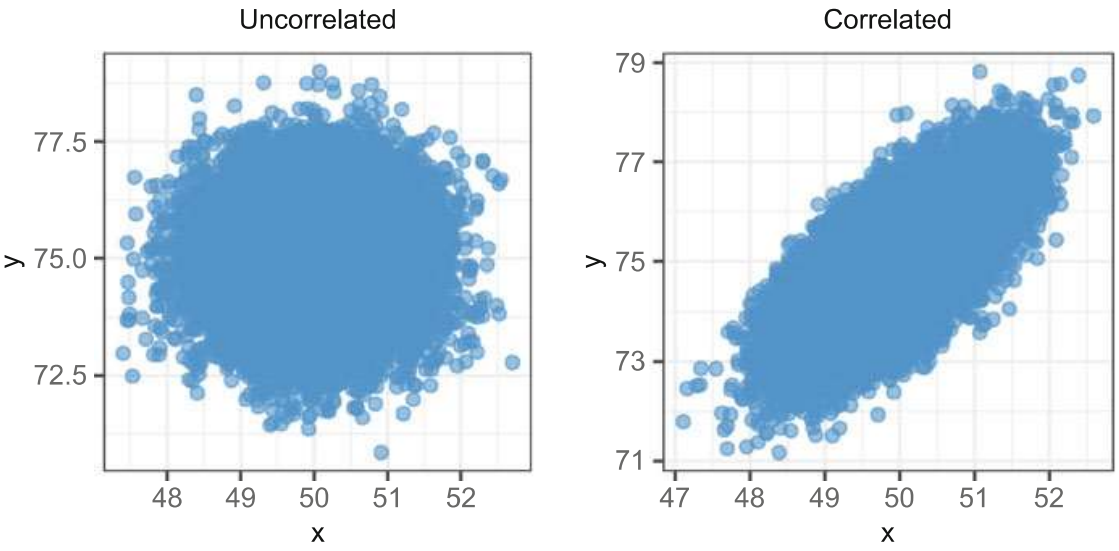


Fig. 8.2 Example scatter plots showing correlated and uncorrelated random samples

8.1.2 Generation of Probability Density Functions Using Random Data

Once the N output values of a Monte Carlo analysis are computed, those samples must be compiled into meaningful statistics. While the output samples are not necessarily normally distributed, it is common to compute the mean and standard deviation of all N samples using the formulas in Chap. 4.

When the output distribution deviates from normal, a more precise description of the distribution is sometimes desired. Plotting a histogram of the samples provides insight into the true distribution. The probability density function of a distribution can be approximated by taking a histogram and connecting the top of each bar to obtain a discrete representation of the data. Changing the binning of the histogram will change the accuracy and smoothness of the result. The KDE method described in Chap. 4 is a more robust method of approximating a PDF from sampled data.

Figure 8.3 shows the histogram of a non-normal distribution with the line showing a KDE approximation of the PDF. While the mean of this distribution is 0.29, the most probable value is about 0.20.

8.1.3 Computational Approaches

This section describes and evaluates some PRNG algorithms and the PRNGs implemented in common programming languages.

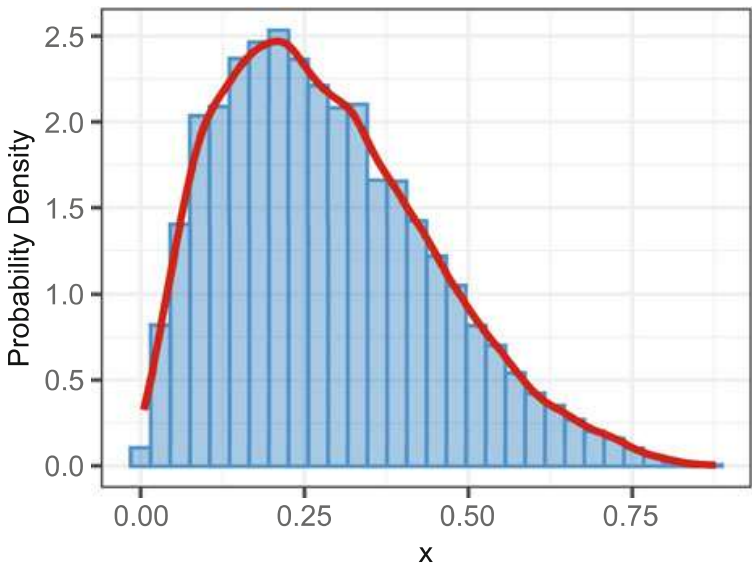


Fig. 8.3 KDE approximation of probability density function

8.1.3.1 Linear Congruential Generator

One of the oldest (if not the best) PRNGs is an algorithm called the Linear Congruential Generator (LCG). While generally not suitable for Monte Carlo analysis, it is described here for its simplicity and as a starting point for understanding other PRNG algorithms. In particular, it provides the basis for the recommended Enhanced Wichmann–Hill PRNG described in the next section. The LCG operates on integers and is implemented by a single iterative equation:

$$X_{n+1} = (aX_n + c) \bmod m, \tag{8.2}$$

where mod is the modulo (remainder) operator.

Three constant parameters are used: a, c, and m. The selection of these constants, and a starting value X_0, called a "seed," determines the sequence of random numbers. The choice of a and c is important for the quality of the random sequence. For example, choosing $a = 4$, $c = 1$, $m = 9$, and an initial value of $X_0 = 0$ leads to a sequence of 0, 1, 5, 3, 4, 8, 6, 7, 2, 0, 1, 5, 3, 4, ... Notice that after nine numbers, the sequence begins to repeat itself. This LCG algorithm with a sequence length of nine (the value of m) is not suitable for generating the few million samples required for typical Monte Carlo methods. In reality the parameters are carefully chosen to be much larger numbers with a large sequence length. For example, using $a = 1664525$, $c = 1013904223$, and $m = 2^{32}$ is recommended by Press et al. (2002). This results in just over 4 billion samples before repeating. Using the sequence length criteria above, this setup is good for about 65,000 samples. In addition, the LCG does not produce uncorrelated values over long sequences (Marsaglia 1968). For these reasons, LCG is generally not suitable for Monte Carlo analysis.

8.1.3.2 Better PRNG Algorithms

Two PRNGs that satisfy the requirements are the Mersenne Twister (Matsumoto and Nishimura 1998) and the Enhanced Wichmann–Hill Generator (Wichmann and Hill 2006), not to be confused with their original 1982 version (Wichmann and Hill 1982). These generators have a large sequence length and pass various randomness tests. While they operate on similar principles as the LCG, they overcome its limitations. The Wichmann–Hill algorithm is simply a combination of four parallel independent LCGs. The Mersenne Twister, instead of using a single seed (X_0 in the LCG) and three constants to record the internal state of the generator, uses 2500 bytes of state. It is also slower and requires much more code to implement.

Typically, one should rely on the PRNG implementation built into a computing package such as R, Python, or Matlab after evaluating it for the above requirements rather than attempting to implement it independently. Most modern programming languages use the Mersenne Twister, but be aware that some older languages, or ones that are slow to update, still use PRNGs unsuitable for the task. If you must

Table 8.1 PRNG implementations in common programming languages

Algorithm	Languages	Sequence length	Maximum samples	Suitable for MC?
Mersenne twister	R, Python, Matlab, Mathematica, C/C++11, Excel >=2010[a]	2^{19937} -1	$\sim 10^{3000}$	Yes
Enhanced Wichmann and Hill (2006)	Available as optional packages in R and Python. Recommended by National Physical Laboratory and GUM.	2^{120}	$\sim 10^{18}$	Yes
Original Wichmann and Hill (1982)	LabVIEW, Excel <=2007[b], Excel VBA (all versions to date)	2^{42}	$\sim 2 \times 10^6$	No
LCG	Visual Basic <= 6	2^{24}	~ 4096	No
LCG	Java	2^{48}	$\sim 16 \times 10^6$	No
LCG	Older C/C++	2^{31}	~ 46,000	No

[a]No seeding is available in Excel. Numbers will be recomputed every update with a different seed, so it fails criteria #4. Excel VBA still uses Original Wichmann–Hill. Only uniform distributions are available directly

[b]Excel 2007 and earlier versions also had errors in the PRNG implementation (McCullough 2008)
References: CRAN (2018), Python Software Foundation (2018), Mathworks Inc. (2018), Wolfram (2018), Brown (2018), Melard (2014), National Instruments (2018), McCullough (2008), Javamex (2018), Barreto & Howland (2006)

implement your own PRNG from scratch, the Enhanced 2006 Wichmann–Hill algorithm as concisely described by Cox and Harris (2006) is recommended for its relative ease of implementation.

See Table 8.1 for a list of some common programming languages, their built-in PRNG algorithm (as of versions released in 2020), and whether they are suitable for Monte Carlo uncertainty propagation based on the above requirements.

Similarly, transformations from uniform to other distributions are typically implemented by each programming language and should be used when available.

8.2 Standard Monte Carlo for Uncertainty Propagation

This section introduces the use of Monte Carlo techniques for quantifying the uncertainty associated with an indirect measurement using a measurement equation, known as the "propagation of distributions" (JCGM 101: 2008).

8.2.1 Monte Carlo Techniques

When applied to uncertainty propagation problems, Monte Carlo methods are straightforward to implement. Figure 8.4 shows a graphical illustration of the Monte Carlo approach to uncertainty propagation. Each input variable is randomly

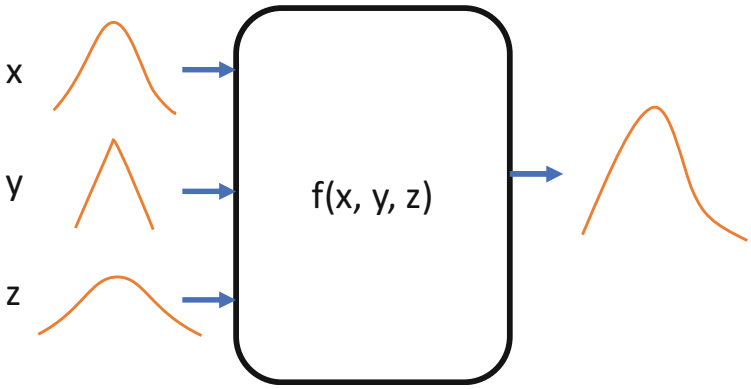


Fig. 8.4 Illustration of Monte Carlo uncertainty propagation

Table 8.2 Measured values for cylinder density calculation

Measurement	Mean	Uncertainty ($k = 1$)
Mass	5 g	0.01 g
Height	2 cm	0.05 cm
Diameter	0.5 cm	0.05 cm

sampled N times from its distribution, and N possible output values are computed using the measurement model. The mean and standard deviation of the output distribution are calculated to provide the expected value and standard uncertainty of the result.

The method is best illustrated with an example.

8.2.1.1 Case Study: Calculating Density

To measure the density of a cylinder, measurements are made on the height h, diameter d, and mass m. The density is calculated using

$$\rho = \frac{m}{\pi\left(\frac{d}{2}\right)^2 h}. \tag{8.3}$$

```
# Measurement equation
rho <- function(m, h, d) {(m/(pi*(d/2)^2*h))}
```

Measured values and their uncertainties (all assumed normal) are given in Table 8.2.

```
# Assign mean values and uncertainties for variables
m <- c(5.00, 0.01)
h <- c(2.00, 0.05)
d <- c(0.50, 0.05)
```

To apply Monte Carlo uncertainty propagation, mass, height, and diameter were each sampled 1,000,000 times from their respective probability distributions.

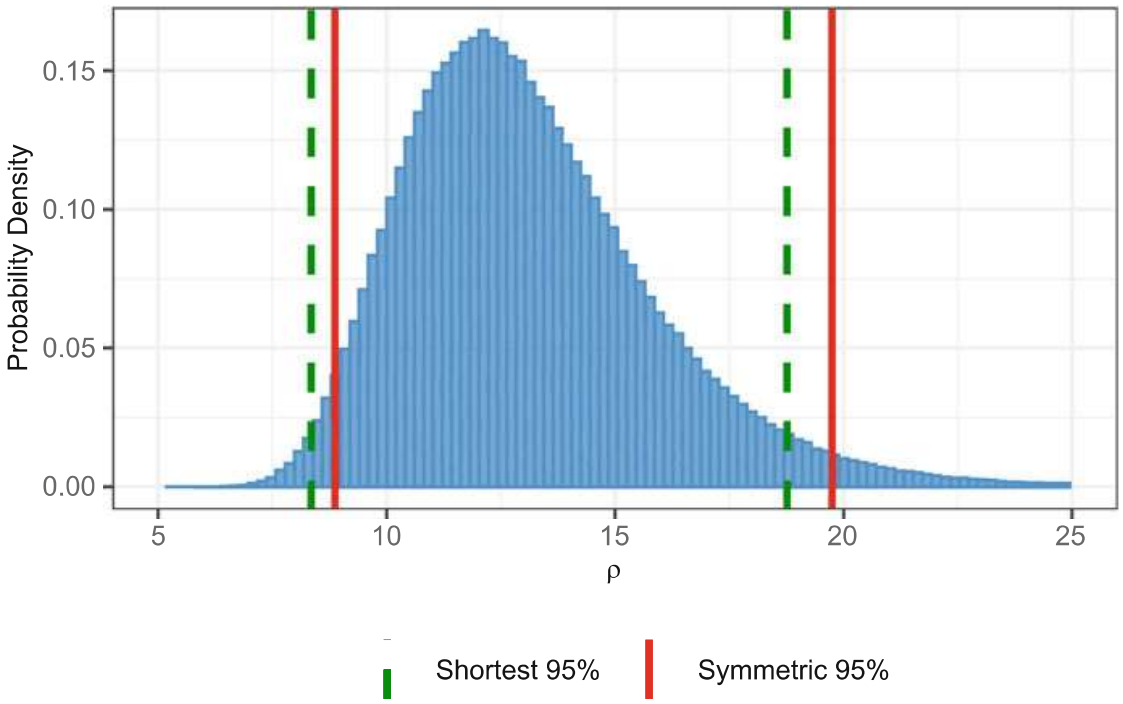


Fig. 8.7 Shortest and symmetric confidence intervals

8.2.1.2 Sensitivity Coefficients

The GUM method calculates sensitivity coefficients from the partial derivatives of the function with respect to each input, which provides information on the relative contribution to total uncertainty of each variable. The Monte Carlo method does not directly provide sensitivity coefficients, although they can still be estimated using the nonlinear model.

The technique for estimating these coefficients is to first hold all variables constant except one. Sample the selected variable N times and determine the uncertainty of the output due to the change in that variable alone. To find the sensitivity coefficient for the input being varied, divide the standard deviation in output due to the single variable by the standard deviation of the variable itself (JCGM 101:2008). This results in a "nonlinear sensitivity coefficient" for that variable (Cox and Harris 2006). If the model is linear, this value will be close to the sensitivity coefficient calculated by the GUM. To find the proportion of total variance, divide the variance in the output due to the single variable by the total variance in the output when all inputs are randomly sampled. The proportion results may not exactly add to 100% due to the random numerical nature of the calculation.

In the cylinder density example, sensitivity coefficients and proportion of total variance are given in Table 8.4. Here, the diameter measurement is by far the biggest contributor to uncertainty in the density.

Table 8.4 Nonlinear sensitivity coefficients

Variable	Sensitivity	Proportion (%)
m	2.55	0.01
h	6.37	1.29
d	55.6	98.41

```
# Calculate sensitivity coefficients
n <- 1e6 # Number of bootstrap samples

# Vary m only
sample <- rnorm(n, m[1], m[2])
mBoot <- rho(m = sample, h = h[1], d = d[1])
sdM <- sd(mBoot)

# Vary h only
sample <- rnorm(n, h[1], h[2])
hBoot <- rho(m = m[1], h = sample, d = d[1])
sdH <- sd(hBoot)

# Vary d only
sample <- rnorm(n, d[1], d[2])
dBoot <- rho(m = m[1], h = h[1], d = sample)
sdD <- sd(dBoot)

sens <- t(data.frame(m = sdM/m[2], h = sdH/h[2], d = sdD/d[2]))

# Calculate proportion
totalVar <- sd(out)
prop <- t(data.frame(m = (sdM/totalVar)^2*100, h = (sdH/totalVar)
^2*100, d = (sdD/totalVar)^2*100))
varResults <- cbind(sens, prop)
colnames(varResults) <- c('Sensitivity', 'Proportion')
varResults
##  Sensitivity  Proportion
## m  2.545723 0.008230318
## h  6.375892 1.290673314
## d  55.599243 98.145899963
```

8.2.1.3 Convergence Plots and Adaptive Sampling

One problem with Monte Carlo methods is the inherent numerical nature of the calculation. For example, if only 10 samples were taken, the output distribution cannot possibly provide enough information to predict the uncertainty range. The Law of Large Numbers says that with infinite samples, the solution will converge

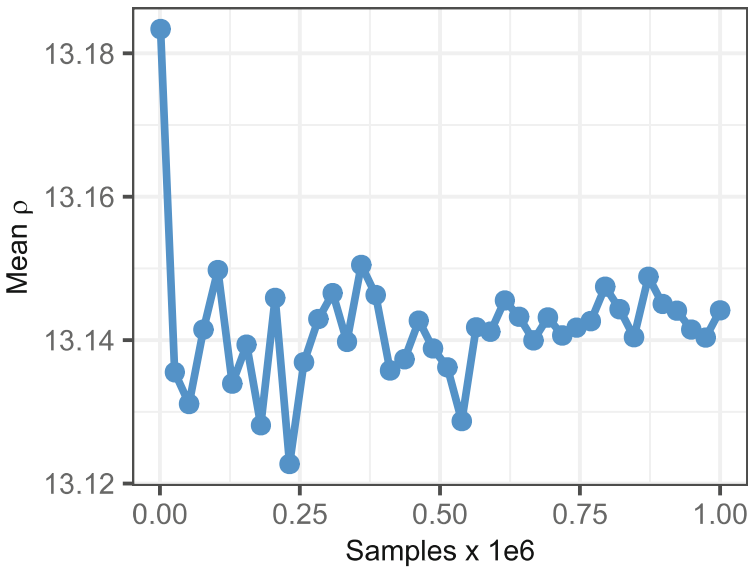


Fig. 8.8 Convergence plot

exactly. Unfortunately, no one has infinite time to solve the problem, so how many samples is enough to be reasonably sure you have a solution?

A convergence plot can help determine if the solution has settled on a "final" value. This plot is generated by gradually increasing the number of samples N, and estimating the unknown parameter for each value. The value of N should be increased until the plot levels off, with little additional change observed by further increasing N.

In the cylinder density example, the solution has reasonably converged after about 500,000 samples (see Fig. 8.8). Similar problems generally require 500,000 to 1,000,000 samples for a solution but occasionally need even more.

Algorithms such as this can be made more formal by continually adding samples until some pre-determined convergence criteria is met. Typically, a tolerance is set and a subset of samples in the neighborhood of 10,000 is computed. A new set of 10,000 samples is computed, and if the two solutions differ by more than the tolerance, the calculation continues by increasing N. The calculation stops once adding a new set of samples does not significantly change the result. Unless the model is quite complicated, modern computers are fast enough that several million samples can be computed in seconds and the additional complexity of adaptive sampling is not worth the effort.

8.3 Comparison to the GUM

The preceding discussion of Monte Carlo uncertainty propagation should make its advantages over the GUM approach apparent. Monte Carlo does not require the Taylor series expansion and makes no assumptions about the linearity of the measurement model. Each input variable can maintain its probability distribution

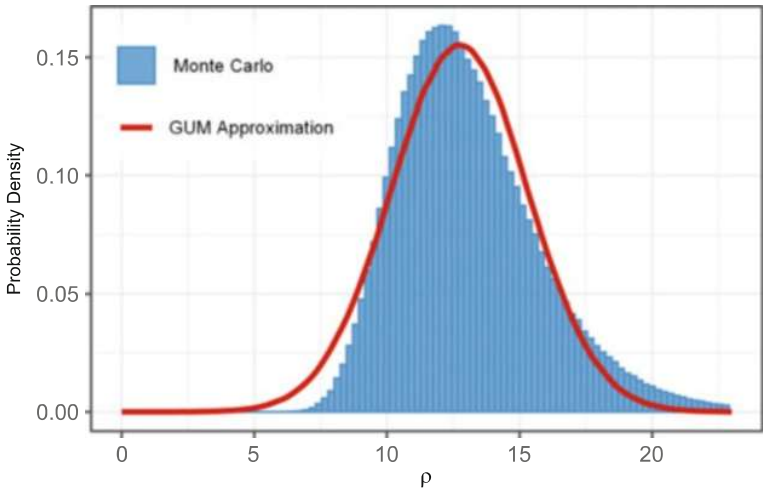


Fig. 8.9 Comparison of Monte Carlo and GUM for uncertainty propagation example

shape through the entire calculation, whereas the GUM requires converting all inputs into normal distributions before computing.

The downsides to Monte Carlo are the computation time required to generate many random samples, the need for quality random number generator algorithms, and the potential variability of the results from one calculation to the next due to the random nature of the algorithm. In addition, the lack of a single analytical expression makes the solution less portable to other similar uncertainty calculation problems based on the same measurement model.

Both GUM and Monte Carlo approaches only approximate the true uncertainty of a measurement model. However, comparing the two results can provide insight into the validity of the approximations. A qualitative comparison can be made by superimposing plots of the GUM resulting probability density function over the Monte Carlo output histogram. Close agreement provides validation of the GUM approximation to the problem.

Figure 8.9 compares the GUM and Monte Carlo methods for the cylinder density uncertainty example. The metrologist will often rely on experience and judgement to decide if the difference is enough to discount the GUM approach.

8.3.1 Quantitative GUM Validity Test

A more quantitative comparison of the GUM and Monte Carlo approaches can be used to validate the GUM approximation (Cox and Harris 2006; JCGM 2008). The approach is:

1. Compute 95% confidence intervals using both the GUM method and the Monte Carlo method.

8.4 Monte Carlo Case Studies

This section provides two practical examples of using Monte Carlo for uncertainty propagation: the neutron yield measurement first introduced in Chapter 6, and the time constant of a resistor-capacitor (RC) circuit.

8.4.1 Case Study: Neutron Yield Measurement

The neutron yield measurement using a lead probe detector was introduced in Sect. 6.4.1 and calculated using the GUM approximation. Here, it is calculated again using the Monte Carlo approach. Recall that the number of neutrons, η, is calculated from

$$\eta = AfF(S - B), \tag{8.6}$$

where A is the calibration factor, f is a conversion factor, and F is the attenuation factor. Variables S and B are the scalar and background counts from the detector. The mean and standard uncertainties for each input variable are repeated in Table 8.6.

```
# Measurement equation
eta <- function(A, f, F, S, B) {A*f*F*(S - B)}
```

```
# Mean values for input quantities and standard uncertainties
f <- 4353
A <- c(1, 0.020)
F <- c(1.27, 0.032)
S <- c(9700, 98.5)
B <- c(80, 2.83)
```

To use the Monte Carlo approach for uncertainty propagation, random samples for each of these variables must be pulled from their respective probability distributions. Variable f is treated as known without uncertainty, so it may be considered a

Table 8.6 Distributions for input quantities and standard uncertainties

Input quantity	Mean (μ)	Standard uncertainty	PDF type A	Standard uncertainty type B	PDF type B
f	4353				
A	1.00			0.020	Uniform
F	1.270			0.032	Uniform
S scalar counts	9700	98.5	Poisson		
B background counts	80	2.83	Normal		

Both limits pass the test, so the GUM result is deemed consistent with the Monte Carlo result to 1 significant digit.

8.4.2 Case Study: RC Circuit

An RC circuit is constructed using commercial off-the-shelf (COTS) components (Fig. 8.14). The components are a 5 kΩ resistor with 1% tolerance, a 0.22 μF capacitor with 5% tolerance, and a parallel 0.1 μF capacitor with 1% tolerance. Because the component uncertainties are given as tolerances, uniform distributions are used.

The time constant of the circuit is found using the measurement model:

$$\tau = R(C_1 + C_2). \tag{8.7}$$

The distribution of 1 million Monte Carlo samples, along with the results calculated by the GUM method, are shown in Fig. 8.15. Note that the GUM method results in a normal distribution whereas the Monte Carlo method produces a more trapezoidal shape, and the 95% limits are not the same. In this case, the difference is not from nonlinearity of the measurement model, but in the GUM's conversion of all inputs into normal distributions.

```
# Measurement equation
tau <- function(R, C1, C2) {R*(C1+C2)}

# Mean values and uncertainties for variables
R <- c(5.00, 0.01)
C1 <- c(0.22, 0.05)
C2 <- c(0.10, 0.01)

# Convert uncertainties to percent and normalize uniform distribution
R[3] <- (R[1]*R[2])/sqrt(3)
C1[3] <- (C1[1]*C1[2])/sqrt(3)
C2[3] <- (C2[1]*C2[2])/sqrt(3)
```

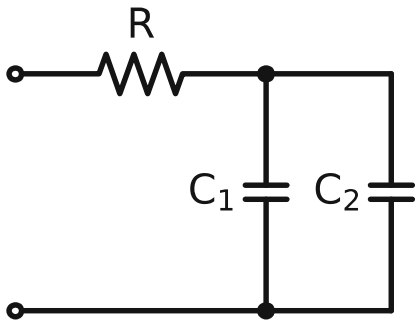


Fig. 8.14 Resistor-capacitor circuit

```
#95% Confidence interval
gum <- rnorm(n, gumMean, uComb)
k <- qnorm((1 - (.05/2)))
gumConfInt <- c(gumMean - k * uComb, gumMean + k* uComb)
gumConfInt
## 1.534936 1.665064

# Generate Monte Carlo Input Samples
n <- 1e6
RSamp <- runif(n, R[1]-R[1]*R[2], R[1]+R[1]*R[2])
C1Samp <- runif(n, C1[1]-C1[1]*C1[2], C1[1]+C1[1]*C1[2])
C2Samp <- runif(n, C2[1]-C2[1]*C2[2], C2[1]+C2[1]*C2[2])

# Generate Monte Carlo Output Samples
out <- tau(R = RSamp, C1 = C1Samp, C2 = C2Samp)

# Symmetric coverage interval
symInterval <- quantile(out, probs = c(0.025, 0.975))
symInterval
##   2.5%  97.5%
## 1.542273 1.658352
```

8.5 Summary

This chapter introduced the Monte Carlo method of uncertainty propagation. The Monte Carlo method uses large sets of random samples to approximate the distribution of possible output values from the measurement model. Because of the numerical nature, it does not require a measurement model that can be written down in closed form. Unlike the GUM, it works in situations where the measurement model cannot be differentiated symbolically, for example, in functions that must be solved iteratively. It also accounts for non-normal input probability distributions and nonlinearities in the measurement model, resulting in an output probability distribution that is not necessarily normal.

It is typically recommended to perform a Monte Carlo uncertainty calculation in parallel with the GUM approximation in order to fully understand the results of both methods.

8.6 Related Reading

Donald Knuth's classic book series *The Art of Computer Programming* is essential reading for any programmer, but Volume 2 (Knuth 1998) is most relevant to the Monte Carlo methods of this chapter. It discusses random number generation in detail, including justification for choice of particular constants in LCGs and various empirical and theoretical statistical tests to ensure randomness in the output. Because Monte Carlo is by nature a parallel computation, it lends itself to parallel implementations such as on a GPU. However, care must be taken to ensure the random numbers are distributed between processes and recompiled correctly. Kindratenko (2014) gives a thorough overview of GPU MC implementation.

The GUM's Supplement 1 (JCGM 101, 2008), also released as ISO/IEC Guide 98-3, details use of Monte Carlo methods for uncertainty propagation. ISO/TR 10017 mentions Monte Carlo as an applicable statistical method but that it should not be used when exact analytical methods are available.

Cox and Harris (2006) and Cox and Siebert (2006) provide guidance on Monte Carlo specifically for uncertainty propagation. They explore convergence criteria, sampling from joint probability distributions for correlated inputs, adaptive sampling, and the multivariate output case where the result is an "uncertainty region" rather than a univariate distribution. Crowder and Moyer (2006) detail an approach to Monte Carlo that accounts for small sample sizes by using a two-stage bootstrap sampling technique. Willink (2007) analyzes the two approaches to extracting coverage intervals from Monte Carlo results (briefly mentioned in Sect. 8.2.1.2). Another example of using Monte Carlo methods is evaluation of uncertainty of weighing in air, including a thorough evaluation of the PRNG used in the evaluation, given by Flicker and Tran (2016).

8.7 Exercises

1. Use your favorite programming language to write an algorithm that estimates π using a Monte Carlo method. Make a convergence plot showing the estimated π value as a function of number of Monte Carlo trials up to 1 million trials.
2. Monte Carlo Methods can be used to approximate definite integrals, and may have advantages over other numeric integration approximations when the dimensionality becomes high. For the function $f(x)$, continuous in the range from a to b, the integral is approximated by:

$$\int_a^b f(x)\,dx \cong \frac{(b-a)}{N}\sum_{i=0}^{N} f(x_i),$$

0, suggesting the "true" value of the function $f(x, y, z)$ is 0. Does the PDF given by the Monte Carlo histogram suggest a true value of 0?

15. In addition to airspeed indication, pitot-static tubes can provide an aircraft with altitude information. The formulas for calculating altitude, h, and velocity, v, are

$$v = \sqrt{\frac{2(p_a - p_d)R_s T_a}{p_a}}$$

$$h = -\frac{R_s}{g_0} T_a \cdot \ln\left(\frac{p_a}{p_0}\right)$$

where R_s is the specific gas constant for dry air (287.06 J/K/kg), g_0 the gravitational acceleration (9.80664 m/s^2), and p_0 the atmospheric pressure measured at sea level. The aircraft measures the ambient temperature T_a, ambient static pressure, p_a, and dynamic pressure p_d (expanding the difference Δp from the previous exercise). The value for p_0 is provided by air traffic control as 101.4 ± 2.1 kPa. Given readings of $T_a = 236 \pm 2.5$ K, $p_a = 21 \pm 0.5$ kPa, and $p_d = 15 \pm 0.5$ kPa, perform a Monte Carlo uncertainty evaluation for v and h, plotting histograms for both. Assume R_s and g_0 have no uncertainty, and all other uncertainties are given at $k = 2$. (With the units given, results will be in meters/second and meters, respectively. Convert to preferred units of the aviation industry, such as knots—nautical miles per hour—and feet, if desired.) Then plot a scatter plot of h versus v. Do the output values appear correlated? Use your software to calculate the correlation coefficient between h and v.

References

Barreto, H., Howland, F.M.: Introductory econometrics using Monte Carlo simulation with microsoft excel. Cambridge University Press, Cambridge (2006)

Box, G.E., Muller, M.E.: A note on the generation of random normal deviates. Ann. Math. Stat. **29** (2), 610–611 (1958)

Brown, W. E.: Random number generation in C++11 (WG21 N3551). Standard C++ Foundation. https://isocpp.org/files/papers/n3551.pdf (2018). Accessed April 28, 2020

Cox, M.G., Harris, P.M.: Software specifications for uncertainty evaluation, DEM-ES-010. Middlesex, UK, National Physical Laboratory (2006)

Cox, M.G., Siebert, B.R.L.: The use of a Monte Carlo method for evaluating uncertainty and expanded uncertainty. Metrologia. **43**(4), S178–S188 (2006)

CRAN Task view: probability distributions. (2018). https://cran.r-project.org/web/views/Distributions.html. Accessed June10, 2018

Crowder, S.V., Moyer, R.D.: A two-stage Monte Carlo approach to the expression of uncertainty with non-linear measurement equation and small sample size. Metrologia. **43**(1), 34–41 (2006)

De Comte Buffon, G.: Essai d'arithmetique morale. In: Supplement a l'histoire naturelle. Nabu Press, Charleston, SC (1774)